

Riwaz Shrestha
DS210

Link to Dataset: <https://catalog.data.gov/dataset/nypd-shooting-incident-data-historic>

- Breakdown of every shooting incident that occurred in NYC
- Each record represents a shooting incident in NYC and includes information about the event, the location and time of occurrence. In addition, information related to suspect and victim demographics is also included.

Purpose:

The purpose of this project was to find the crime hotspots in New York's boroughs. I also filter through victim sex and victim race to find relationships within each category. I used the NYPD Shooting Incident Data (Historic), which contains data from 2006 to 2013. For my purposes and to keep the project somewhat small, I used only the geolocation, victim sex, and victim race. I ran out of energy and the will to do more, but I plan on making another map based on race, plotting a different color per race. I would also like to do more with the time of day and perpetrator data.

GPT Usage:

I resorted to using ChatGPT quite a bit for this project. I did not use GPT for the data processing module. For my K Means module, I used GPT to create closures and help me with the structure of how to create each iteration. I asked for no code and to just give me a structure. I heavily used GPT for the Visualizations module. I am not too familiar with the Plotter's library, so I had Chat create a template for me to input all the necessary information. I also had it help me create the PNG as the output. For my main function, I used GPT to troubleshoot some errors I was having. The most notable contribution would be the error handling and explaining all the errors I ran into. For my test functions, I used GPT to bounce my errors and figure out how to use assert.

Data Processing:

In this module, I process each given row and return a struct containing the information needed for the rest of the program, this being the geolocation, victim sex, and victim race. In the process function, I split the row by commas, skip over the first index, make sure the number of columns is correct, and then assign each variable in the struct to an index in the vector, assigning string or f64 accordingly.

K Means Clustering:

I first create the point and cluster structs. The point is just x,y as f64. The cluster struct is a point for the cluster and a vector of points for the points that belong to each cluster. I created a distance fn for the points to measure distance. In the K Means function, I create k random centroids and store them in a vector of points. I used GPT to help me with the closure. I then loop through the number of iterations and assign each point to a cluster by pushing them into the cluster, update the centroid, and check if the centroid is converged in each iteration. If

converged, I clear the cluster, push the finalized centroids into a vector, and push their corresponding points to the vector in each cluster.

Visualizations:

I found the geolocation for each borough by hand. I had GPT make me a simple template to use the Plotter's library. This is the template it gave me:

```
use plotters::prelude::*;
use plotters::coord::types::RangedCoordf64;

const PLOT_RES: (u32, u32) = (800, 600); // Width x Height

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // Sample points: (x = longitude, y = latitude)
    let points = vec![
        (-73.9857, 40.7484), // Example: NYC Empire State Building
        (-74.0060, 40.7128), // NYC
        (-73.9712, 40.7831), // Central Park
    ];

    // Create an image file to draw on
    let root = BitMapBackend::new("simple_plot.png", PLOT_RES).into_drawing_area();
    root.fill(&WHITE)?;

    // Create the chart with coordinate bounds
    let mut chart = ChartBuilder::on(&root)
        .caption("Point Plot Example", ("sans-serif", 30))
        .margin(10)
        .x_label_area_size(30)
        .y_label_area_size(30)
        .build_cartesian_2d(-74.1f64..-73.9f64, 40.6f64..40.8f64)?;

    // Draw the coordinate grid
    chart.configure_mesh()
        .x_desc("Longitude")
        .y_desc("Latitude")
        .draw()?;

    // Plot the points
    chart.draw_series(points.iter().map(|&(x, y)| {
        Circle::new((x, y), 5, RED.filled())
    }))?;

    // Save the file
```

```
root.present()?;  
println!("Plot saved to simple_plot.png");  
  
Ok()  
}
```

For the most part, I just filled in the blanks and adjusted each variable until I got the desired output. One regret is that I ran into many errors trying to label each borough, so I had to give up on that. My `draw_borough` function required me to use GPT to figure out lifetimes.

Main:

In this module, I handled errors by having the main function return a result enum. I imported all my other modules and their functions, along with some resources from the std library. I first get the CSV file, separate the lines, and then call my process data function on each line. For the output of the function, I use an if-let statement to push the required info to “all_points”, “male_points”, “female_points”, and “race counts”. Each of these vectors and hashmap were needed for my graphs and table of victims by race. The vectors are self-explanatory, and for the hashmap, I keep the race as the keys and update the values by adding 1. I finally ran my K-means function using all of the points. I then call all of my graph functions to create the graph PNGs.

Lastly, I print the cluster information to get a better understanding of the graphs (metadata) and print the victims by race table. This was easy, as all I had to do was print a hashmap.

Tests:

For tests, I run K-means on a smaller dataset and check the number of clusters, make sure that no cluster is empty, and make sure that the distance is reasonable (each point is close to a centroid).

My second test makes smaller testing clusters for my plotting functions and makes sure that they do not return errors. I had to have GPT help me with the syntax for this test. I kept getting testing errors and eventually turned to chat to fix the issues. I used Chat mostly to figure out how to use `assert!` and `assert_eq!`.

Outputs:

Should only take a few seconds to run the full project.

Borough boundaries plot saved to `borough_boundaries.png`
All points plot saved to `all_points.png`
Centroids plot saved to `centroids.png`
Complete visualization saved to `complete_visualization.png`

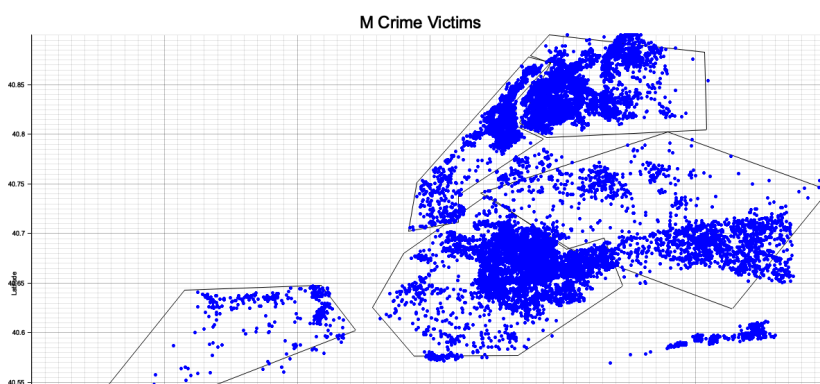
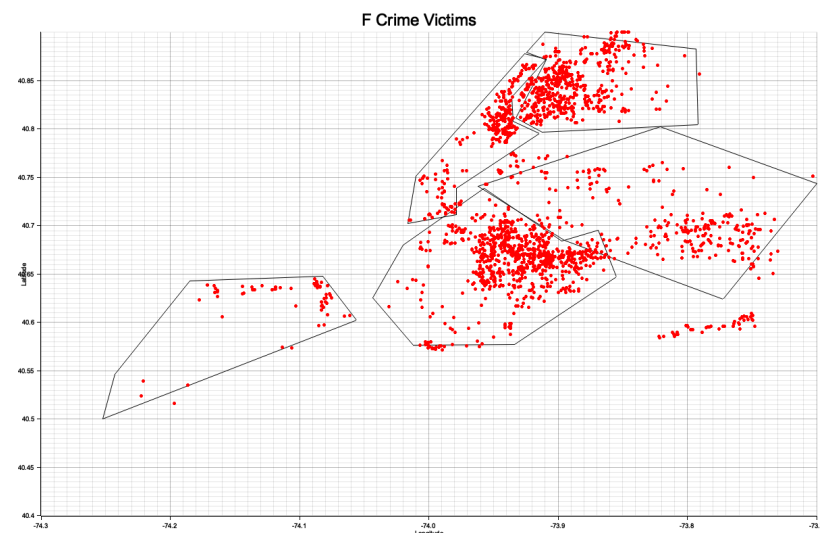
M victims plot saved to m_victims.png
F victims plot saved to f_victims.png

Cluster Information:

Cluster 0: 4184 points
Centroid: (-73.937907, 40.793213)
Cluster 1: 3389 points
Centroid: (-73.785888, 40.674125)
Cluster 2: 2113 points
Centroid: (-73.855635, 40.868918)
Cluster 3: 12473 points
Centroid: (-73.944957, 40.664382)
Cluster 4: 6344 points
Centroid: (-73.903219, 40.839412)

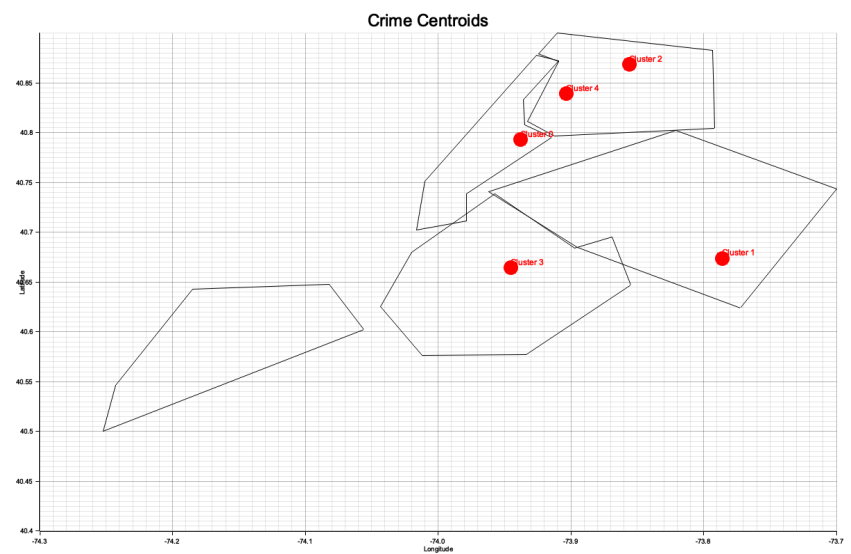
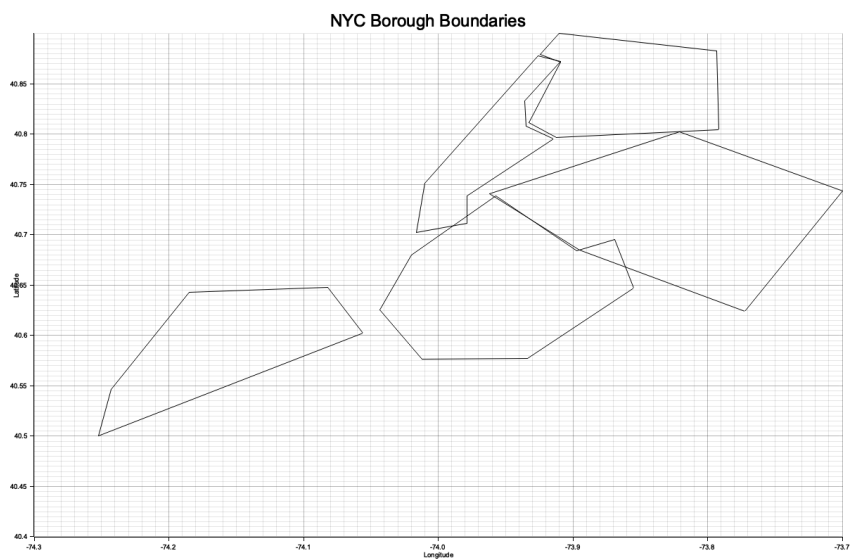
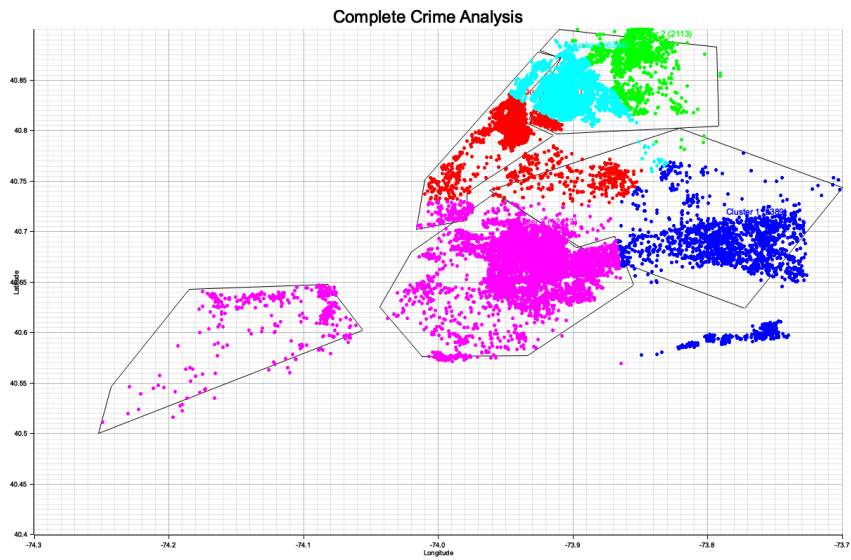
Victim Race Statistics:

AMERICAN INDIAN/ALASKAN NATIVE: 11
ASIAN / PACIFIC ISLANDER: 440
WHITE: 728
BLACK: 20202 (The black population is at most risk)
WHITE HISPANIC: 4265
BLACK HISPANIC: 2787
UNKNOWN: 70



There are a lot more male victims than female victims, but they are generally in the same locations.

There are a lot less shootings in staten Island





There is a lot of crime in New York. Most of them are generally in the middle of the boroughs or very close to cross-borough borders.